Multiplayer

How I approached building a backend I had not built before, what the server owns and what it refuses to tell the client, and the one design decision I made early that shaped the rest.

BRYCE WALTERS MULTIPLAYER.PDF SEPTEMBER 2026

How I approached it

I approached the multiplayer backend with the deliberate intention of letting Claude Code take the lead. I have built out some backend features before, but never a full backend from scratch. Because of that I chose a fairly simple multiplayer game, on the reasoning that the patterns it uses are well known and therefore easy for Claude Code, or any AI agent, to extract and extrapolate to fit the experience I was building.

What I asked for was a well known framework: the Model of the MVC lives on the backend, and the frontend controls showing the UI elements and fielding user inputs, which are brought back to the server to determine how to handle them — claiming an item on the conveyor belt, for instance. The server determines who claims what, the coins that appear in each player's hands, how to evaluate whether a player's selection was correct, and which tray items should leave the conveyor belt.

After Claude built out the boilerplate backend code to support multiplayer, I reviewed it to make sure I understood what was happening — again using Claude Code to validate my understanding of the code rather than just reading it and hoping.

What the server owns

Everything that decides the game. The client can say one thing about play: that a particular player wants a particular tray. It cannot say that it bought it, or what the score now is. The server resolves the claim, updates the game, and tells every browser in the room what happened.

THE SERVER DECIDES	THE CLIENT DOES
Which coins are in each player's hand, and what that hand is worth	Draws the coins it is given
Which tray is baked next, and at what price	Draws the tray and its price
Where every tray is on the belt	Animates the move between the two positions it was told
Who claimed a tray first, and whether they were right	Sends the tap and waits
Which trays leave the belt, and whether that counts as waste	Plays the tray falling
The score, the target, and whether the game is won or lost	Renders the two meters

One consequence is worth stating on its own, because it is the whole reason this game is a teaching tool rather than a clicking exercise. **A hand's value never crosses the wire.** The server sends the list of coins a player is

holding and keeps the sum to itself — it is not sent even to the player holding the hand. Working out what the coins are worth is the exercise, so the answer is not available anywhere in the browser to be read out of the developer tools.

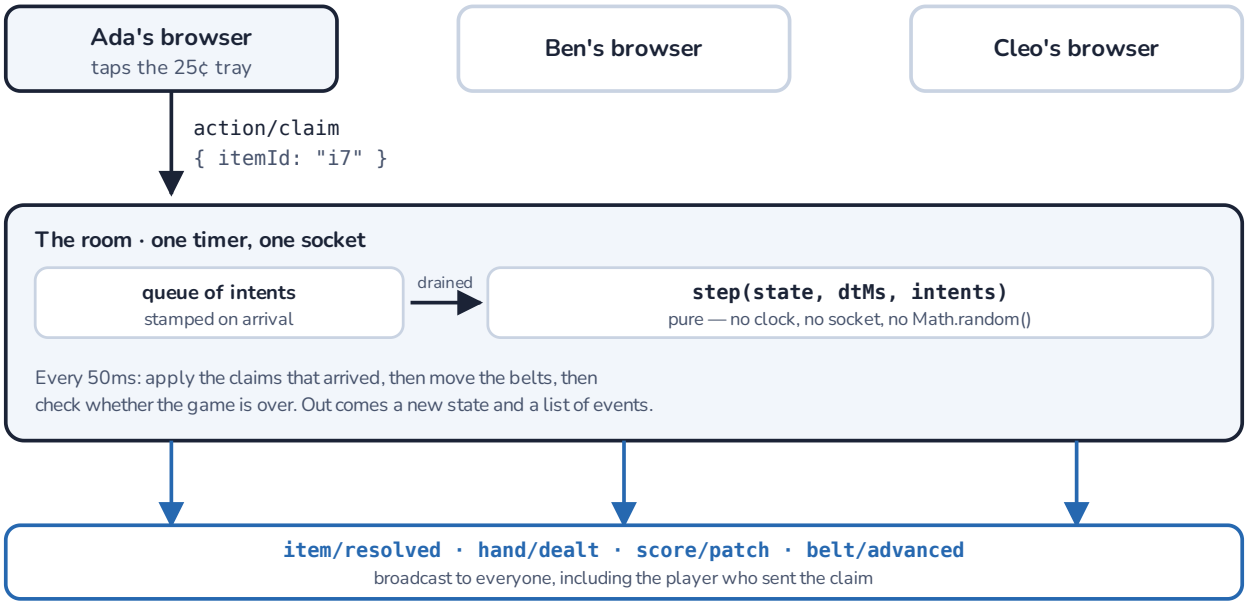
The belts move in steps

One design decision I made early on was to limit the movement of the conveyor belts to steps rather than continuous motion. This was to simplify the experience, and to avoid needing a high refresh rate that could cause some students to become out of sync.

In the code a belt is an array of eight slots, and a tray is in a slot or it is not. On a beat — every three seconds on easy, faster on the harder settings — the server shifts the array one place towards the incinerator and sends the new occupancy to every client. The server never computes a pixel, it quotes the client a slot pitch as advice and the client maps a slot index into its own geometry. The visible hop is decoration played out over about a quarter of a second, and the tray then rests for the remainder of the interval.

That resting period turned out to be worth more than the bandwidth it saved. A second grader aiming at a stationary tray is aiming at a target rather than leading a moving one, and "which tray did you tap" has an unambiguous answer at every moment. It also means there is nothing to interpolate and nothing to predict: a client that misses a message and asks for a fresh snapshot is immediately correct again, rather than having to be smoothly corrected.

A claim, end to end



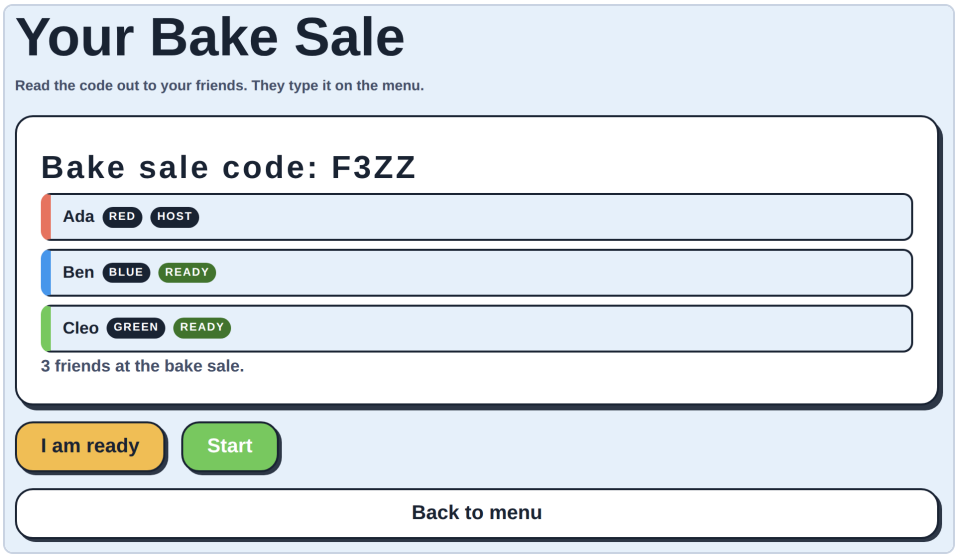
One tap, one answer. Claims are queued as they arrive and resolved together at the top of a tick, so "who grabbed it first" has a single answer rather than being a race between two socket callbacks. Ties inside the same millisecond break by player id — deterministic, and not winnable by lying about your clock.

All of the rules live in one function, `step(state, dtMs, intents)`: it takes the current game, how much time has passed, and the intents that arrived, and returns the next game plus the list of events to send out. It has no

clock, no sockets and no randomness of its own — time arrives as a number and chance comes from a seed carried inside the state. One file owns the only real timer and the only real socket in the server.

That shape is what makes the rest of it testable, and it is a large part of why I was comfortable letting an agent build it. The rules can be exercised with no network at all, and a whole game replays deterministically from a seed and a list of intents, so a complaint that a spawn looked unfair becomes a test with a seed number in it.

Rooms and joining



The lobby. The host reads the code out; the others type it on the menu. Colours are assigned by the server in join order, so the first child in a fresh room is always red.

A room is a four-letter code and an entry in a map in memory. There is no database, no Redis and nothing persisted — restart the process and every bake sale is gone, which is the intended behaviour for a game that lasts a few minutes. Codes are issued by the server rather than generated by the client, because only the server can promise the code is not already taken.

Identity is disposable and scoped to the room. There are no accounts: a child types a name and a code and is in. A resume token stored in the browser tab lets a player who drops out rebind to the seat they already hold rather than arriving as a stranger. The first player through the door is the host, their only privilege is pressing Start, and if they leave it passes to the longest-connected player still there — which is cheap precisely because the server is authoritative, so nothing in the game depends on who the host is.

A few smaller decisions in the same area, all of which exist because of how children actually use this: an empty room is kept for thirty seconds rather than dropped instantly, so a host who refreshes does not destroy the code they just read out to three friends; a socket that stops answering a heartbeat is closed, because a tablet that went into a bag does not report itself as gone; and in the lobby a disconnected player's seat is released, while mid-game it is kept and greyed out, because the target score is calculated from the number of players and must not move under the children still playing.

The dealer

This is the part of the backend that is not boilerplate, and it does two jobs.

The first is keeping the game solvable. No player should be left holding a sum that nothing on the belt matches. Whenever the oven has a free slot, the hand that has gone longest without a matching tray takes priority over anything else, so nobody can be starved by a run of unlucky rolls. Hands are also dealt with distinct values across the players where the range allows, so two children are not sent racing for the same tray.

The second is that the trays which are *not* matches are priced deliberately. A decoy is priced at a plausible wrong answer for one of the hands in play: the hand's value plus or minus a coin that is actually in it, the number of coins instead of what they are worth, the digits transposed, or the whole pile counted as though every coin were the same denomination. Those come from the same error taxonomy the lesson uses to diagnose a wrong answer.

So when a child grabs the wrong tray, the mistake is classified — against the hand they were holding, not against the label the tray was baked with — and the end of the game reports per-child error counts in the same vocabulary the lesson reports in. The game produces the lesson's diagnostic signal without needing a separate assessment.

> A wrong grab wastes the tray and puts that player on a short cooldown, but does not replace their hand. Dealing a new hand would let a child escape a sum they could not do by guessing at it, which is the opposite of what the game is for.

Reviewing what was built

The rules being a pure function is also what made them checkable. The server has its own test suite — fifty-four tests across ten suites, none of which opens a socket — and the suite names are a fair summary of what I wanted to be sure of.

```
belt motion              claiming                winning and losing
hands                    decoys                  classifying a wrong grab
what crosses the wire    the lobby               starting a game
solvability

# tests 54   # pass 54   # fail 0
```

Two of those matter more than the rest to me. *What crosses the wire* is the check that a hand's value never leaves the server. *Solvability* is the check that a player is never left holding a value the bakery refuses to bake for.

What is not built

The difficulty table describes three belts, incinerator doors that cycle open and closed, and belts that jam when the oven has nowhere to put a tray. The shipped game pins itself to one belt with the doors held open: the full rule is implemented and one function overrides it, so turning it on is a deletion rather than a feature. Reconnecting after a longer drop, pausing a game, and a bot client for testing a room single-handed are also not built.